

Sw12 - Feature matching and image Registration

Recap: Detection of key points

- Key points (or points of interest, salient points) are points in the image with distinctive characteristics
- Examples: Harris corners, SIFT or SURF features
- Ideally, key points are robust to changes in the image, such as rotation, scaling, translation, noise, illumination changes, etc.
- Key points are used in various computer vision tasks, such as image registration, tracking, stereo vision, and object recognition.

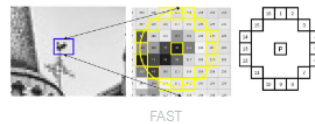


Find the differences... Which key points (Shi-Tomasi features) are stable?

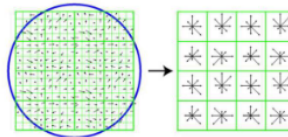
Feature descriptor

- Some key point detection methods come with a **descriptor** that describe the image in the proximity of the feature location
- Concretely: A descriptor is a set of numbers that describes the region around a key point (like appearance, shape, or texture)
- **Idea:** Use descriptors to compare key points with respect to similarity.
- Descriptors usually come with their own distance metric. Often, this is simply the Euclidean norm of the difference between two descriptors:

$$\|\vec{d}_a - \vec{d}_b\| = \sqrt{(d_{a,0} - d_{b,0})^2 + (d_{a,1} - d_{b,1})^2 + \dots}$$



FAST

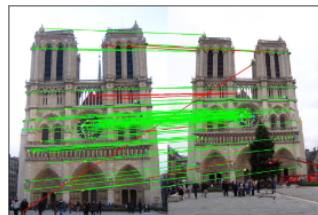


SIFT

Illustrations of different descriptors. Top: FAST feature descriptor. Bottom: SIFT feature descriptor. Details are omitted here

Feature matching

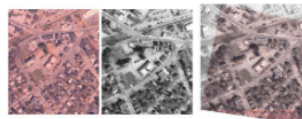
- Feature matching is the process of finding corresponding points in two images.
- Steps:
 1. Detect key points in both images
 2. Compute the descriptors for the key points
 3. Match the descriptors
- Wrong matches (often called outliers) can occur due to:
 - Occlusions or perspective changes
 - Illumination changes
 - Noise (or other distortions)



Examples for matching features in two images. Matches are visualized using a green connecting line. Red dots (and lines) identify non- or mis-matching features. Both examples rely on SIFT features.

Image registration

- Image registration: Use matching feature pairs to **estimate the transformation between two images**.
- Transformation: Mathematical rule (function) with which we can **project a pixel p of image I_1 into I_2**
- Subtypes of image registration:
 - Rigid registration, using
 - similarity transform: Translation, rotation, and scaling
 - affine transform: Translation, rotation, scaling, and shearing
 - projective transform: Perspective transformations
 - Non-rigid registration:
 - Makes use of local deformations
 - Transformation fields



Examples for rigid / affine transforms

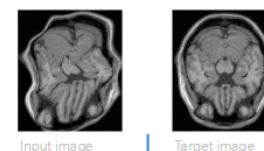
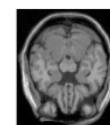


Image registration

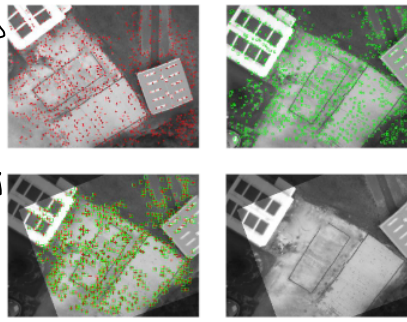


Result

Example for non-rigid transform

Steps:

1. Feature matching
 - Detect key points in both images
 - Compute the descriptors
 - Match the descriptors
2. Estimate the transformation $T : p \rightarrow q$
 - Often involves solving an optimization problem
 - Variables: Parameters θ of the transformation
 - Input data: Matched pairs of key points: (p_i, q_i)
 - Cost function: mean reprojection error, i.e. distance between q_i and the transformed points $p_i : q_i = T(p_i)$
 - Minimization of cost function yields optimal θ
3. Apply the transformation to the image



Top: Left image containing key points p_i , and right image containing features q_i . Bottom: matched and projected features after estimation of the (affine) transformation T

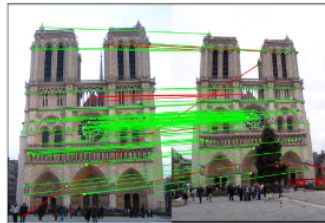
Image registration: RANSAC

- How to detect outliers?
- Popular method: RANSAC
 - RANSAC: Random sample consensus
 - Iterative method to estimate the parameters of a mathematical model.

Steps:

1. Select a subset of data points randomly
2. Fit the model to the selected data points
3. Compute the error of the model on all data points
4. Count the number of inliers (= points that are consistent with the model)
5. Repeat steps 1-4 N times, or until convergence

- Extension: PROSAC (Progressive sample consensus). ... 😊



Note the non-matching pairs of points. They have a negative effect on the estimated transformation T

Illustration of the RANSAC algorithm for a line model: two points are sampled randomly, and we track which of the models finds best support in the dataset.
Link

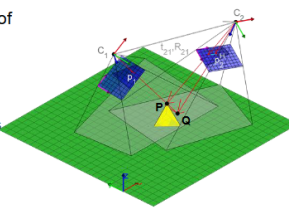


Applications for Feature matching

- Aerial image registration (assumption, all points lie approx. in one plane, camera should be further away from the ground)
- Stereo vision: comparing perspectives to get 3D information

Application: Stereo vision

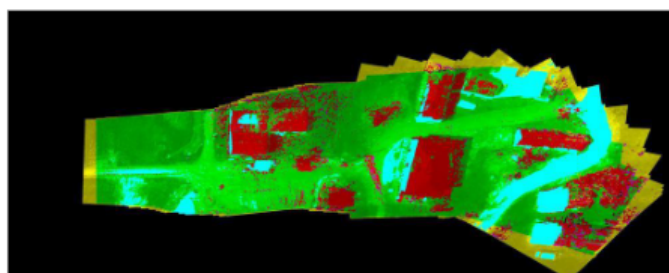
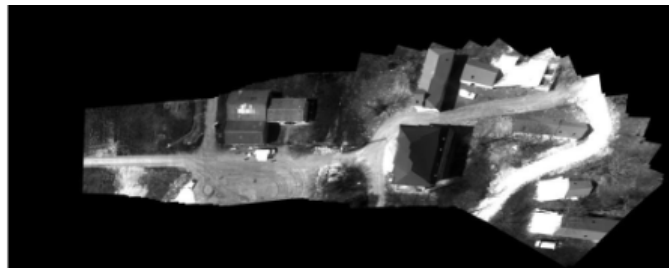
- Goal (of stereo vision): Compute the depth of objects in a scene from two (or more) images.
- Depth map: An image that encodes the depth of objects in a scene for every pixel.
- Typical steps:
 - Align the images, for example by matching features.
 - Estimate the disparity (displacement) for each pixel.
 - Compute the depth map from the estimated disparities



- and Panorama feature of cameras: Image stitching / Panography
↳ wide angle panoramic images from multiple images

• Traversability mapping:

- Stereo vision can be used to assess ground traversability for a ground vehicle.
- Input: Sequence of aerial images
- Steps:
 - Detect and match features
 - Align the images
 - Compute the depth map (estimate depth at each pixel).
 - Threshold the depth map to identify obstacles and free space.
- Output:
 - A map indicating passable areas for cars, etc.



Sav12 - Segmentation

It's about classification

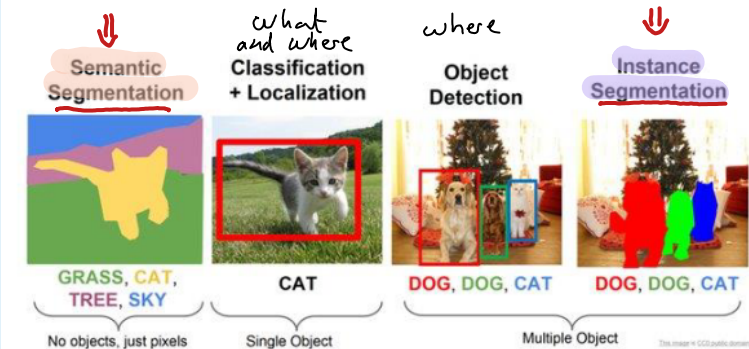
Levels:

- Subject-level: healthy or not
- Image level: indicate an image a pathological condition or not
- Patch-level: classify pathological status of a local region
- Pixel-level: assign a class label to each pixel (segmentation?)

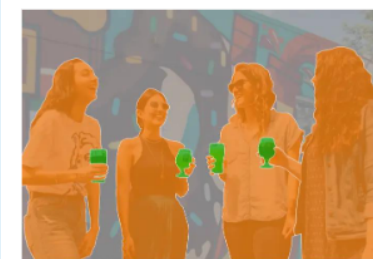
Segmentation is the process of partitioning an image into distinct, non-overlapping regions

Types of approach:

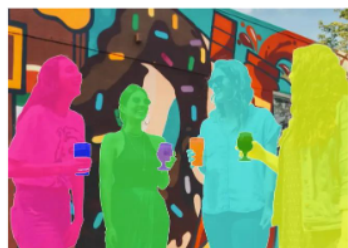
1. Image processing / image analysis (rule-based, model-driven methods)
2. Machine learning (learning patterns from data)



Both Segmentation



Example for semantic segmentation. All pixels in the image have a label, same objects have the same label.



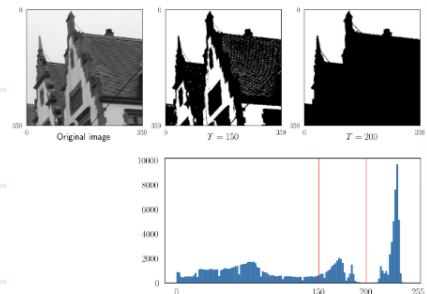
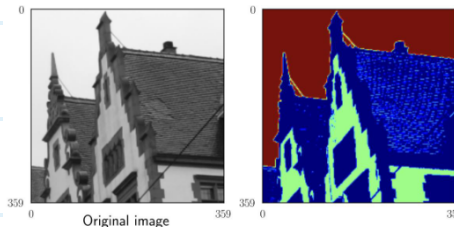
Example for instance segmentation. Instances of the same object have different labels (but still, objects of the same class can be grouped together)

Segment anything Model (SAM)

- Deep-learning-based image segmentation method by Meta AI
- Foundation Model for different Task
- Generates segmentation masks from simple prompts
- Pretrained Model is trained on Billion Masks from 11M images
- Zero-shot capability: Segment new objects and domain without retraining
- Open-source with released model weights for broad integration

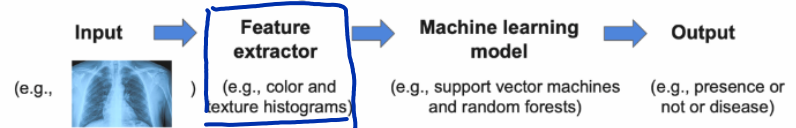


Multiple Thresholds:

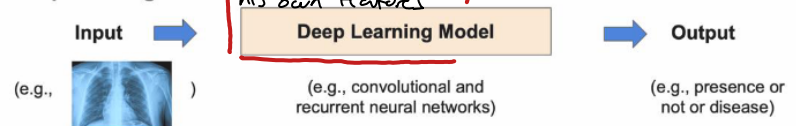


How to approach classifications problems?

Traditional machine learning



Deep learning

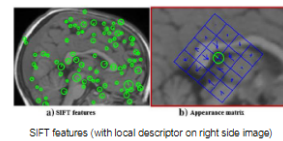
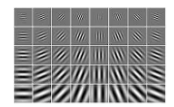
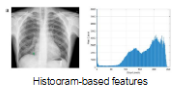


Two types of feature extraction:

1) Feature extraction for classification: Traditional approach

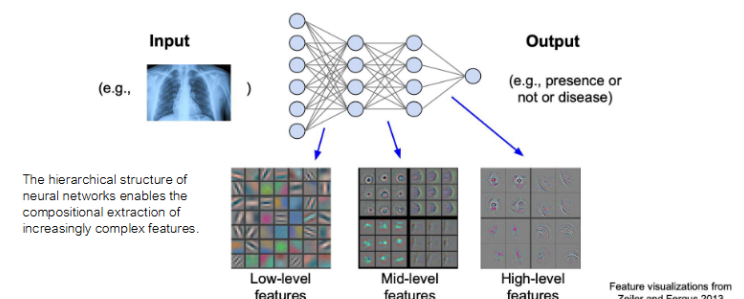
Types of features (non-exhaustive list):

- Intensity-based features
 - Histogram-based / statistical features (mean, std, moments, skewness, kurtosis)
 - Gradient-derived features
- Texture-based features
 - Gray-level co-occurrence matrix features
 - Gabor filter responses
- Shape-based features
 - Geometric features (area, size ratios, perimeter, etc...)
 - Fourier descriptors
- Local descriptors
 - SIFT (Scale-Invariant Feature Transform)
 - SURF (Speeded Up Robust Features)
 - Harris corner detector
 - HOG (Histogram of Oriented Gradients)



SIFT features (with local descriptor on right side image)

2) Feature extraction: Deep learning



Simplest form of image segmentation

=> Binary thresholding

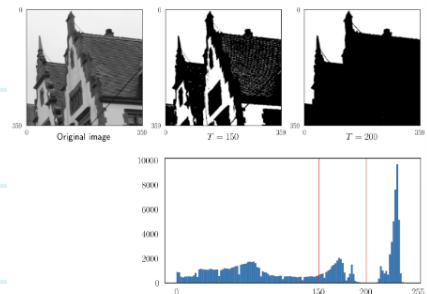
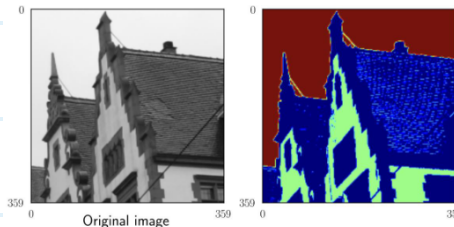
- Partitioning an image into foreground and background by comparing pixel intensities to a fixed threshold

Input: Grayscale $I(x, y)$

Parameter: T (local or global) => Threshold

$$B(x, y) = \begin{cases} 1, & \text{if } I(x, y) \geq T \\ 0, & \text{if } I(x, y) < T \end{cases}$$

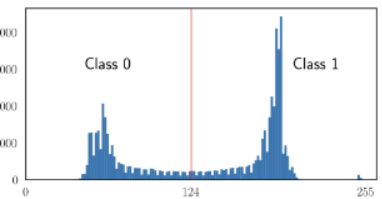
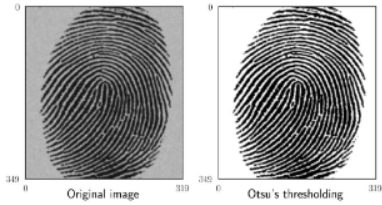
Multiple Thresholds:



Binary thresholding - Otsu's method

Find the optimal threshold value T that separates image pixels into two classes by minimizing intra-class variance

- Works best if the image has a bimodal histogram



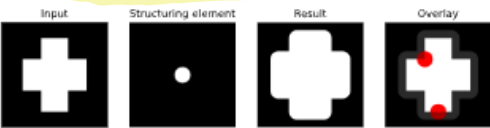
Morphological operations

- Non-linear operator that act on images based on local spatial structure
- Typically applied to binary images (grayscale)

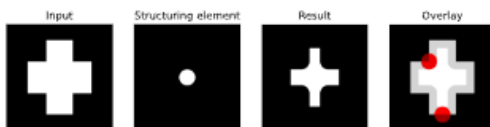
Core idea:

1. use a small pattern called a structure element (SE)
2. Slide it over an image
3. Modify pixels based on how the SE fits or overlaps

Dilation operation Expands foreground regions by adding pixels to object boundaries. A structuring element slides over the image, and a pixel becomes foreground if it overlaps with any existing foreground pixel, causing objects to grow and small gaps to close.



Erosion operation Here, in contrast, foreground regions are shrunk by removing pixels at object boundaries. A pixel remains foreground only if the structuring element fits entirely within the object, causing objects to contract and small structures to disappear.

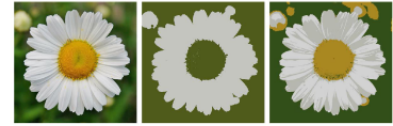
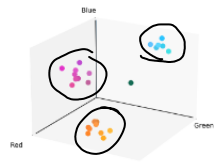


useful for post-processing segmentation masks

- Remove noise (op. opening)
- Fill holes in objects segmentation (op. closing)
- Separate touching objects (op. erosion)
- Recover object size after cleaning (op. dilation)

Color clustering

- Idea: Segment an image by grouping pixels with similar values / colors into clusters.
- Approach: Pixels are represented in a color space (e.g., RGB, HSV, Lab) and clustered based on color similarity.
- Common method: k-means clustering
- Advantages:
 - Simple and easy to implement
 - Unsupervised (no labeled data required)
- Disadvantages:
 - The number of clusters must be specified *a priori*
 - Sensitive to initialization and parameter choices



k-means clustering

1. Initialize
 - Randomly initialize K cluster centroids.
2. Iterate until convergence:
 - Step A (Assignment): For each data point
 - Compute its distance to each centroid
 - Assign the point to the nearest cluster
 - Step B (Update): Recompute each centroid as the mean of all points assigned to that cluster

